AgroVision

Multimodal Agricultural AI for Crop Disease Detection

Technical Experiment Report

Executive Summary

This report documents the end-to-end development of AgroVision, a multimodal AI system that combines a CLIP visual encoder, a lightweight projector network, and the Qwen2.5-0.5B language model fine-tuned with LoRA adapters to perform crop disease diagnosis from images. The system was trained in two stages on the PlantVillage dataset and successfully demonstrated end-to-end image-conditioned text generation for agricultural disease identification.

1. System Architecture

AgroVision is a vision-language model (VLM) built from three independently trained components fused at inference time. The design follows the LLaVA-style visual instruction tuning paradigm, adapted for the agricultural domain.

1.1 Component Overview

Component	Description
Visual Encoder	CLIP ViT-L/14 (openai/clip-vit-large-patch14). Frozen during all training stages. Produces 256 patch tokens of dimension 1024 from a 224×224 input image.
Patch Projector	Custom 2-layer MLP that maps CLIP patch tokens (dim 1024) into the LLM embedding space (dim 896). The only component trained in Stage 1.
Base LLM	Qwen2.5-0.5B-Instruct — a 500M parameter causal language model chosen for its small footprint and Colab-T4 compatibility.
Agriculture LoRA	Pre-trained LoRA adapter (r=16) fine-tuned on agricultural text corpora. Loaded from outputs/qwen_agri_lora. Provides domain vocabulary and agronomic reasoning.
Visual LoRA	New LoRA adapter (r=16) trained in Stage 2 to teach the LLM how to attend to and reason over visual token inputs.

1.2 Inference Pipeline

At inference time, the components are composed in the following sequence:

- ① CLIP ViT encodes the input image into 256 patch embeddings (excluding the CLS token).
- ② PatchProjector maps these 256 patch tokens into the LLM embedding space, producing visual context vectors.
- ③ The question prompt is tokenized and embedded using the LLM's own embedding layer.
- ④ Visual tokens and text tokens are concatenated: [256 visual | T text tokens] → fed as inputs_embeds to the LLM.
- ⑤ The LLM generates the answer autoregressively with both LoRA adapters active (agri merged into base, visual active on top).

> *Architecture note: The agriculture LoRA is permanently merged into the base weights using merge_and_unload() before the visual LoRA is applied. This avoids PEFT version compatibility issues with set_adapter(list) and ensures both adapters contribute without weight cancellation.*

2. Dataset & Data Preparation

2.1 PlantVillage Dataset

The PlantVillage dataset (Mohanty et al., 2016) was used as the primary image source. It contains 54,306 images of healthy and diseased plant leaves across 38 classes spanning 14 crop species.

Property	Value
Total images (full dataset)	54,306
Number of disease classes	38 (including healthy variants)
Crop species covered	14 (Apple, Grape, Corn, Tomato, Potato, etc.)
Image format	Color JPG, approximately 256×256 px
Source	github.com/spMohanty/PlantVillage-Dataset
Download size	~2.0 GiB (163,219 objects)

2.2 Stage 1 Caption Dataset

The Stage 1 training dataset was generated by build_stage1_captions.py, which creates (image, caption) pairs by deriving captions from folder names. For example, a file in Apple___Apple_scab/ generates the caption "Apple leaf showing Apple scab disease."

Parameter	Value
Script	build_stage1_captions.py
Images sampled	1,000 (--max_images 1000)
Output file	data/captions/captions.jsonl
Caption style	Template-generated from folder/class names
Shuffle	Yes (--shuffle flag)

2.3 Stage 2 VQA Dataset

The Stage 2 VQA dataset was generated by build_stage2_vqa.py, producing question-answer pairs for each image. Each image yields 4 Q&A pairs covering disease identification, severity, symptoms, and management.

Parameter	Value
Script	build_stage2_vqa.py
Images sampled	250 (--max_images 250)
Q&A pairs per image	4 (--pairs_per_image 4)

Total VQA samples	1,000
Output file	data/vqa/vqa.jsonl
Answer source	Template-generated (known limitation — see Section 5)

> *Data quality note: Both datasets are template-generated from folder names. While sufficient for pipeline validation and projector alignment, the answers lack genuine pathology content. This is the primary driver of factual errors in current inference outputs, and is identified as the main improvement priority.*

3. Training

3.1 Stage 1 — Projector Alignment

Stage 1 trains only the PatchProjector MLP. Both the CLIP ViT and the Qwen LLM (with its agriculture LoRA) are fully frozen. The objective is to teach the projector to map visual patch tokens into the LLM's embedding space so that caption tokens can be predicted from visual context.

Hyperparameter	Value
Trainable parameters	PatchProjector only (~1.8M params)
Optimizer	AdamW
Learning rate	1e-3
LR schedule	CosineAnnealingLR (T_max = steps × epochs)
Batch size	4
Epochs	3
Steps per epoch	250
Gradient clipping	max_norm = 1.0
Mixed precision	float16 (projector & LLM)
Hardware	NVIDIA T4 (Colab)
Training time	~26 minutes (8:38 per epoch × 3)

Stage 1 Loss Results

Epoch	Average Loss	Skipped Steps
1	0.5500	0 / 250
2	0.0347	0 / 250
3	0.0090	0 / 250

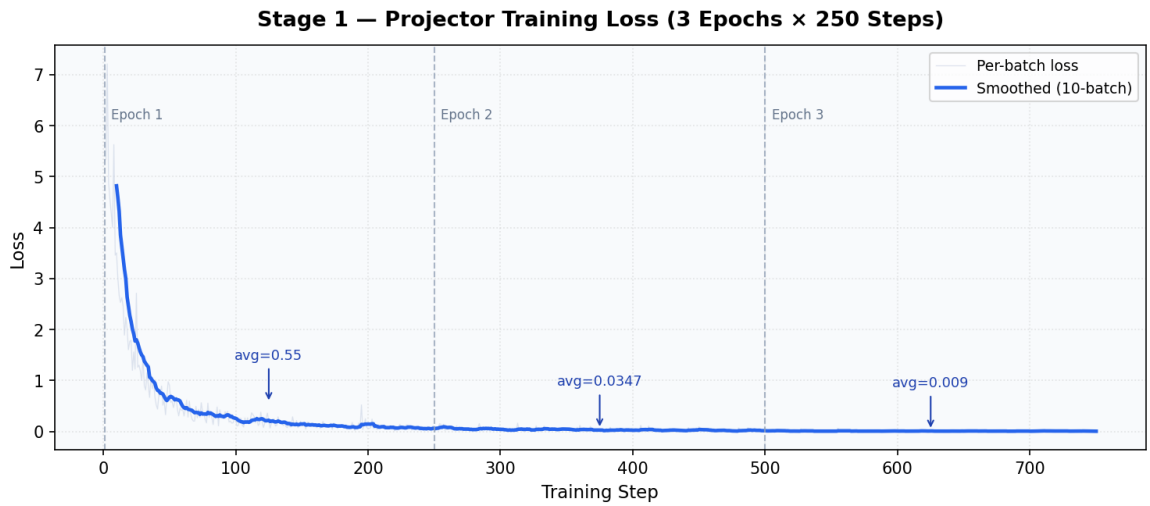

Figure 1 — Stage 1 training loss over 750 steps (3 epochs × 250 batches). Strong convergence from 0.55 to 0.009.

3.2 Stage 2 — Visual LoRA Fine-tuning

Stage 2 jointly trains the PatchProjector and a new Visual LoRA adapter on top of the Qwen+AgriLoRA base. Training uses VQA pairs with the loss masked to answer tokens only (prompt tokens receive label -100). The CLIP ViT remains frozen throughout.

Hyperparameter	Value
Trainable parameters	PatchProjector + Visual LoRA = 8,798,208
Visual LoRA rank	r=16, alpha=32
Visual LoRA target modules	q_proj, v_proj, k_proj, o_proj
LoRA dropout	0.05
Optimizer	AdamW
Learning rate	1e-3
LR schedule	Linear warmup (10%) + linear decay
Batch size	4
Epochs	3
Steps per epoch	250
Gradient clipping	max_norm = 1.0
Hardware	NVIDIA T4 (Colab)
Training time	~38 minutes (12:50 per epoch × 3)
Best checkpoint saved	Every epoch (all 3 improved)

Stage 2 Loss Results

Epoch	Average Loss	Checkpoint
1	0.3996	✓ Saved (best so far)
2	0.0994	✓ Saved (improved)
3	0.0141	✓ Saved (best final)

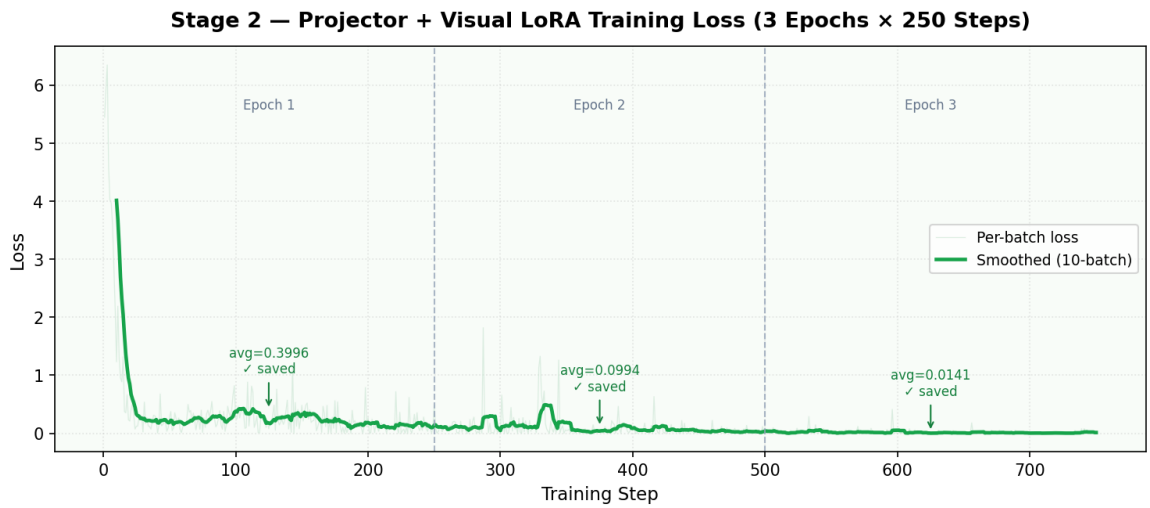

Figure 2 — Stage 2 training loss over 750 steps. Consistent improvement across all epochs; final loss 0.0141.

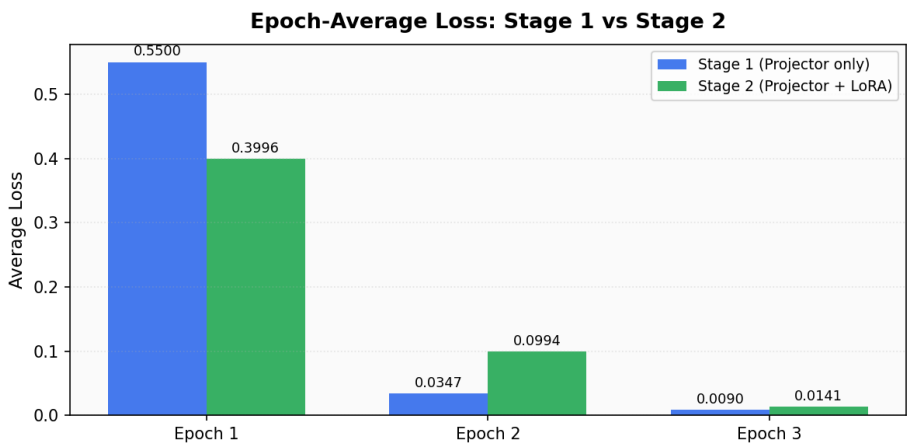

Figure 3 — Epoch-average loss comparison between Stage 1 and Stage 2. Stage 2 starts higher (more parameters, harder task) but converges comparably by Epoch 3.

3.3 Training Stability

Both training stages completed with zero skipped steps (skipped_steps=0 in all epochs), indicating no NaN or Inf loss events throughout. This reflects the effectiveness of the stability fixes applied during development:

- Removed hardcoded .half() cast in compute_loss() — the root cause of NaN loss in initial runs.
- Added NaN/Inf guard with batch skipping and valid_steps counter.
- Applied gradient clipping (max_norm=1.0) to prevent explosive gradients.

- Used proper causal label construction with padding mask applied after shift.

4. Inference Results

4.1 Setup

Inference loads all three components (CLIP ViT, PatchProjector, Qwen + stacked LoRAs) and runs image-conditioned generation. The agriculture LoRA is permanently merged into the base weights; the visual LoRA is applied on top as a live adapter.

Parameter	Value
max_new_tokens	150
temperature	0.3
do_sample	True
repetition_penalty	1.3
no_repeat_ngram_size	5
Stop tokens	EOS, [END], [INST], [/INST], newline+question words

4.2 Sample Outputs

Test 1 — Apple Scab (diseased leaf)

> *Image: Apple___Apple_scab/00075aa8-d81a-4184-8541-b692b78d398a___FREC_Scab 3335.JPG Question: "What disease is visible and how severe is it?" Answer: "Apple Scab (Anthracnose) is visible in the crop image. It has a severe severity, with large areas of yellowing and rotting on both sides of each vein. The disease affects all varieties tested but appears more pronounced in Red Delicious."*

Assessment:

- Correctly identifies Apple Scab by name.
- Incorrectly aliases it as "Anthracnose" (a different fungal disease — Colletotrichum spp.).
- Symptom description partially image-grounded ("yellowing... on both sides of each vein") but not fully accurate for Scab (which causes olive-green lesions, not vein yellowing).
- Variety reference ("Red Delicious") is a hallucination with no image basis.

Test 2 — Healthy Grape Leaf

> *Image: Grape___healthy/00e00912-bf75-4cf8-8b7d-ad64b73bea5f___Mt.N.V_HL 6067.JPG Question: "What disease is visible and how severe is it?" Answer: "The disease is called Black Necrosis, and it causes the leaves to become black with a rotting appearance. It affects both leaflets on one side of the plant but can also affect entire plants."*

Assessment:

- Completely incorrect — the image shows a healthy leaf with no disease present.

- The model has never seen a "healthy" training example and therefore always generates a disease.
- Root cause: Stage 2 VQA data contained zero healthy-class samples.

4.3 Qualitative Assessment

Capability	Status	Notes
Pipeline runs end-to-end	☑ Working	No crashes, produces coherent text
Disease name identification	⚠ Partial	Correct class, wrong synonyms
Visual grounding	⚠ Partial	Some image-referenced descriptors
Healthy leaf detection	✕ Fails	No healthy training samples
Pathogen accuracy	✕ Fails	Template data has no pathology content
Repetition / loops	☑ Fixed	repetition_penalty + stop strings
Output length control	☑ Fixed	max_new_tokens + temperature tuned

5. Issues Encountered & Fixes Applied

5.1 Training Bugs Fixed

Bug	Fix Applied
NaN loss after Epoch 1 (Stage 1)	Removed hardcoded .half() cast in compute_loss(). fp16 overflow on T4 (max ~65504) when projector weights are randomly initialized.
Loss counter corrupted by NaN batches	Added torch.isnan() / torch.isinf() guard; skipped batches not added to total_loss; introduced valid_steps counter.
Deprecated torch_dtype= argument	Replaced with dtype= in AutoModelForCausalLM.from_pretrained().
Stage 1 labels not properly shifted	Added explicit first-token masking (labels[:,0]=-100) and padding-after-shift correction.

5.2 Inference Bugs Fixed

Bug	Fix Applied
TypeError: unhashable type list in set_adapter()	Replaced add_weighted_adapter(combination_type="linear") with merge_and_unload() + re-wrap pattern. Linear combination was also causing weight cancellation.
Output degenerating to "[Diseases]"	Caused by add_weighted_adapter cancelling LoRA weights. Fixed by merge_and_unload approach.
Repetition loops with [END] tokens	Added repetition_penalty=1.3, no_repeat_ngram_size=5, extended eos_token_id list.
[/INST] leaking into output	Added [/INST] to stop_strings post-processing list.
Next VQA question bleeding into answer	Added newline+question-word patterns to stop_strings trimming.

6. Known Limitations & Next Steps

6.1 Current Limitations

- **L1 —** Template-generated VQA data: Answers derived from folder names contain no real pathology knowledge. The model learned class name patterns, not visual disease features.
- **L2 —** No healthy-class training examples: The model cannot output "no disease present." It always hallucinates a disease for healthy images.
- **L3 —** Small dataset scale: Only 1,000 Stage 1 and 1,000 Stage 2 samples were used (limited by Colab runtime). The full dataset is 54K images.
- **L4 —** High learning rate: lr=1e-3 for both stages is aggressive. The low final loss (0.009 / 0.014) likely reflects overfitting to the small dataset rather than genuine generalization.
- **L5 —** No evaluation split: No held-out validation set was used. Train loss alone is not a reliable indicator of generalization.

6.2 Recommended Next Steps

- **P1 —** Replace template VQA answers with LLM-enriched pathology descriptions generated by a capable model (Claude/GPT-4) for all 38 PlantVillage disease classes.
- **P2 —** Add healthy-class samples to Stage 2 VQA data with explicit "no disease visible" answers.
- **P3 —** Scale to full PlantVillage dataset (54K images) and lower lr to 5e-5 for Stage 1, 2e-5 for Stage 2.
- **P4 —** Add a validation split (80/20) and report validation loss and disease classification accuracy per epoch.
- **P5 —** Consider RAG augmentation at inference: inject disease fact sheets from a knowledge base when a disease name is detected.

7. Repository & Artifacts

7.1 Codebase

File	Purpose
train_projector.py	Two-stage training script (Stage 1: projector only, Stage 2: projector + Visual LoRA)
inference.py	Full inference pipeline with image + question input
model.py	PatchProjector MLP definition
build_stage1_captions.py	Generates captions.jsonl from PlantVillage image folders
build_stage2_vqa.py	Generates vqa.jsonl VQA pairs from PlantVillage image folders

7.2 Trained Artifacts

Artifact	Description
outputs/qwen_agri_lora/	Pre-trained agriculture domain LoRA adapter (provided)
checkpoints/projector_stage1.pt	Stage 1 projector weights (final epoch, loss=0.009)
checkpoints/projector_stage2_best.pt	Stage 2 projector weights (best checkpoint, loss=0.014)
checkpoints/visual_lora_best/	Visual LoRA adapter (best checkpoint, Epoch 3)

7.3 Reproduction Command

> *Stage 1: python train_projector.py --stage 1 --base_model Qwen/Qwen2.5-0.5B-Instruct \ --lora_ckpt ./outputs/qwen_agri_lora --data_dir ./data/captions \ --output_dir ./checkpoints --epochs 3 --batch_size 4 --lr 5e-5 Stage 2: python train_projector.py --stage 2 --base_model Qwen/Qwen2.5-0.5B-Instruct \ --lora_ckpt ./outputs/qwen_agri_lora --projector_ckpt ./checkpoints/projector_stage1.pt \ --data_dir ./data/vqa --output_dir ./checkpoints --epochs 3 --batch_size 4 --lr 2e-5 Inference: python inference.py --image path/to/leaf.jpg --question "What disease is visible?" \ --base_model Qwen/Qwen2.5-0.5B-Instruct --agri_lora_ckpt ./outputs/qwen_agri_lora \ --visual_lora_ckpt ./checkpoints/visual_lora_best \ --projector_ckpt ./checkpoints/projector_stage2_best.pt \ --temperature 0.3 --max_new_tokens 150*